

Разрабатываю compiler infrastructure, где решения о composition, IR/runtime и корректности выражены явно: LLVM-анализы и passes, детерминированное планирование language platform, Bytecode/AIR/SSA, backends и независимые verifier/oracle проверки

LLVM analysis

LICM и межпроцедурные линейные summaries глобальных значений; неподдерживаемые CFG/call cases дают conservative unknown

Language infrastructure

UniversalToolchain: один immutable LanguagePlan владеет dependencies, capabilities, pass ordering и backend routes

Independent verification

Interpreter/CIL parity, exact package binding, allocator verifier и differential execution против reference interpreter

ОПЫТ

1 июля — 31 августа 2026

МЦСТ — стажёр по разработке компиляторов

- Реализовал LLVM LICM-pass и развиваю межпроцедурный анализ эволюции глобальных значений: линейные summaries komponуются там, где это доказуемо; unsupported loops/calls/CFG переходят в unknown
- Проверяю преобразования через opt/lli, verifier и negative cases, где pass обязан сохранить исходный IR или отказаться от вывода

2024 — сейчас

UniversalToolchain — создатель и основной разработчик

- Сделал LanguageCompiler единственным владельцем composition: dependencies/conflicts, capability selection, pass ordering и backend routes фиксируются до runtime в immutable LanguagePlan
- Разделил syntax → semantic binding → Bytecode → AIR → optimizations → backend явными representation boundaries; архитектурные contracts защищены exact binding и regression tests, текущий exact manifest — 1 306/1 306 passed

2026

x86-64 Codegen & Register Allocation Playground

- Отделил allocation strategy от correctness policy: allocators выдают единый Assignment, который независимо проверяется по liveness/interference invariants перед codegen
- Сравниваю native x86-64 execution с reference interpreter в отдельном resource-bounded процессе

ПРОЕКТЫ

UniversalToolchain / Wist2

Текущий exact test manifest проходит без падений; проверяются interpreter/CIL parity, clean-consumer сценарии и воспроизводимые compiler/runtime контракты.

x86-64 Codegen & Register Allocation Lab

Измеряемый pipeline с метриками spill/reload, размером кода и воспроизводимым сравнением результатов.

PS-form Memory Dependence Analyzer

Консервативный результат yes/no/maybe с независимым exact oracle на малых областях, randomized/metamorphic проверками и воспроизводимыми контрпримерами.

КОМПЕТЕНЦИИ

LLVM / Program Analysis

C++23, LLVM passes, LoopInfo, CFG, dominance, SSA, interprocedural summaries, conservative analysis

Compiler Infrastructure

typed language packages, deterministic planning, capabilities/conflicts, pass ordering, exact runtime binding

IR / Runtime / Backend

NET, Bytecode/AIR, SSA routes, interpreter, CIL backend, register allocation, x86-64

Verification / Tooling

reference oracles, differential/metamorphic tests, verifier-gated transforms, CMake, Linux, ASan/UBSan

ОБРАЗОВАНИЕ

Студент программы «Программная инженерия» НИУ ВШЭ

ДОСТИЖЕНИЯ

Диплом I степени и Главная премия «Совершенство как надежда» за UniversalToolchain

Москва / удалённо